foreword

Functional Programming in Scala is an intriguing title. After all, Scala is generally called a functional programming language and there are dozens of books about Scala on the market. Are all these other books missing the functional aspects of the language? To answer the question it's instructive to dig a bit deeper. What is *functional programming*? For me, it's simply an alias for "programming with functions," that is, a programming style that puts the focus on the functions in a program. What are *functions*? Here, we find a larger spectrum of definitions. While one definition often admits functions that may have side effects in addition to returning a result, pure functional programming restricts functions to be as they are in mathematics: binary relations that map arguments to results.

Scala is an impure functional programming language in that it admits impure as well as pure functions, and in that it does not try to distinguish between these categories by using different syntax or giving them different types. It shares this property with most other functional languages. It would be nice if we could distinguish pure and impure functions in Scala, but I believe we have not yet found a way to do so that is lightweight and flexible enough to be added to Scala without hesitation.

To be sure, Scala programmers are generally encouraged to use pure functions. Side effects such as mutation, I/O, or use of exceptions are not ruled out, and they can indeed come in quite handy sometimes, be it for reasons of interoperability, efficiency, or convenience. But overusing side effects is generally not considered good style by experts. Nevertheless, since impure programs are possible and even convenient to write in Scala, there is a temptation for programmers coming from a more imperative background to keep their style and not make the necessary effort to adapt to the functional mindset. In fact, it's quite possible to write Scala as if it were Java without the semicolons.

So to properly learn functional programming in Scala, should one make a detour via a pure functional language such as Haskell? Any argument in favor of this

approach has been severely weakened by the appearance of *Functional Programming in Scala*.

What Paul and Rúnar do, put simply, is treat Scala as a pure functional programming language. Mutable variables, exceptions, classical input/output, and all other traces of impurity are eliminated. If you wonder how one can write useful programs without any of these conveniences, you need to read the book. Building up from first principles and extending all the way to incremental input and output, they demonstrate that, indeed, one can express every concept using only pure functions. And they show that it is not only possible, but that it also leads to beautiful code and deep insights into the nature of computation.

The book is challenging, both because it demands attention to detail and because it might challenge the way you think about programming. By reading the book and doing the recommended exercises, you will develop a better appreciation of what pure functional programming is, what it can express, and what its benefits are.

What I particularly liked about the book is that it is self-contained. It starts with the simplest possible expressions and every abstraction is explained in detail before further abstractions are built on them in turn. In a sense, the book develops an alternative Scala universe, where mutable state does not exist and all functions are pure. Commonly used Scala libraries tend to deviate a bit from this ideal; often they are based on a partly imperative implementation with a (mostly) functional interface. That Scala allows the encapsulation of mutable state in a functional interface is, in my opinion, one of its strengths. But it is a capability that is also often misused. If you find yourself using it too often, *Functional Programming in Scala* is a powerful antidote.

MARTIN ODERSKY
CREATOR OF SCALA